

Session 1

Foundations and the AI service layer

What you build today: `nw/llm`, the client every later project calls a model through.

The three walls every project hits

Wall	What it looks like	What we build
Fan-out	500 prompts, 500 connections, then 429s	bounded concurrency
Unreliable upstream	batch dies at prompt 312	provider-aware retries
Unvalidated output	<code>json.loads</code> raises on "Sure, here is"	structured output with repair

Solve them once. Sessions 2 to 6 import the result.

Where the time goes

- A model call is network I/O: hundreds of milliseconds to seconds of waiting, almost no local compute
- So: `asyncio`, one `httpx` pool per process, a semaphore for the provider's rate limit
- Threads would work. Processes waste memory on idle interpreters
- The semaphore number is a business number: it comes from the provider's quota and how many services share the account

Retryable and terminal are different failures

Status	Family	Reason
429, 529, 5xx, timeout	retryable	transient, the provider's or the network's problem
400, 401, 403, 404	terminal	your request; it will fail the same way again
refusal	terminal	the content, not the call

Retrying a 400 spends money on a certain failure. Not retrying a 429 loses work that would have succeeded in a second.

Backoff with full jitter

- Ceiling doubles per attempt: $\text{base} * 2^{(\text{attempt} - 1)}$, capped
- Delay is uniform between 0 and the ceiling
- Without jitter every client that failed together retries together, and the provider sees the same spike
- `Retry-After` overrides all of it. The provider chose that number

Structured output is a contract

1. Tell the model the JSON Schema of the Pydantic class
2. Parse, then validate. Never trust formatting
3. On failure, send the error back once with the reply
4. Second failure: raise

A loop that keeps asking is a budget leak. One repair fixes most malformed replies in practice.

Why the neutral subset

- Every vendor offers chat, tool calls, token usage, JSON in text
- Vendor-specific structured output is faster on some. It is also the first thing that breaks on a vendor change
- Validation in your own code survives the change
- Vendor features appear in the guide as "try this too", never in the acceptance criteria

Cost is an output, not an afterthought

- Every completion priced from a table, recorded with its correlation ID
- Reserve before the call, replace with the real cost after
- A hard cap that refuses the call, not a dashboard that notices later
- Session 6 turns these numbers into cost per resolved ticket

Model roles, not model IDs

Role	Used in	Today
Workhorse	Sessions 1, 4, 5, 6	Claude Sonnet 5
Judge	evaluation harness	Claude Opus 5
Economy	Session 6 router	Claude Haiku 4.5

Code says `ModelRole.WORKHORSE`. Configuration says which model. Swapping a vendor is one provider file and a config change.

One provider interface, two tracks

- `LLMProvider.complete(messages, model, system, tools, max_tokens)` returns a `Completion`
- Bedrock and Vertex differ only in how the client is constructed
- The fake provider scripts outcomes and counts in-flight calls; that is how the tests prove the semaphore
- Nothing outside `nw/llm/providers/` imports a vendor SDK

The correlation ID

- A context variable set once per request
- Follows the async task through every await without being passed around
- On every log line, on every cost record, and from Session 2 on every trace span
- When a customer asks "what happened to my ticket", this is how you answer

Agent-written code, same bar

- Install the skill set today, use it every session
- `/tdd` to start, `/code-review` before the tests, `/diagnosing-bugs` when it spins, `/session-check` to finish
- Nothing enters a project because an agent produced it. It passes the tests or it does not go in

Acceptance criteria

1. 100 prompts at `max_concurrency=8` never exceed 8 in flight
2. Two 429s then success: succeeds. A 400: fails at once, one attempt
3. `structured` recovers from one malformed reply and raises after the second
4. `map` preserves input order under out-of-order replies
5. Total spend and tokens are queryable, and the cap is a hard stop

`make session01` reports 24 passed.

Questions worth asking

- What is the right concurrency for a provider that allows 50 requests a minute, shared by two services?
- When is the repair round trip more expensive than failing fast?
- Which of today's failures would have reached production in your current setup?